

DEVOPS PROJECTS

10-Project Cloud, CI/CD & DevSecOps Portfolio

Documentation with screenshots — based on the supplied source ZIP

Projects	PROJECT-1 through PROJECT-10
Primary areas	Docker, Jenkins, GitHub Actions, AWS, CI/CD, Terraform, Ansible, DevSecOps
Cloud services	S3, CloudFront, EC2, DynamoDB, Lambda, API Gateway, CodePipeline, CodeBuild, CodeDeploy
Security / monitoring	Trivy, IAM, Ansible Vault, Prometheus, Grafana, cAdvisor
Source basis	DevOps_Projects-main(1).zip

DevOps Projects — Source ZIP Overview

10 projects included in DevOps_Projects-main

PROJECT-1	Weather App — Dockerized Flask + Nginx
PROJECT-2	Node.js + MongoDB with Docker & Docker Compose
PROJECT-3	Jenkins CI/CD Pipeline with Docker & Node.js
PROJECT-4	AWS CI/CD Node.js Application
PROJECT-5	Flask + Docker + GitHub Actions CI/CD
PROJECT-6	Static Website — AWS S3 + CloudFront + CI/CD
PROJECT-7	AWS DynamoDB Flask Task Manager
PROJECT-8	Serverless CI/CD Pipeline using AWS Lambda
PROJECT-9	Next.js Portfolio — DevOps Ready
PROJECT-10	DevSecOps CI/CD with Terraform, Ansible & Jenkins

Figure 1 — Project index generated from the supplied source tree.

PROJECT-1: Weather App — Dockerized Flask + Nginx

Flask weather application served through Nginx with Docker Compose. It fetches real-time weather data from OpenWeatherMap.

Technology stack

Flask • Nginx • Docker • Docker Compose • OpenWeatherMap

Documented workflow / setup

- Create an .env file with the OpenWeatherMap API key.
- Build and run with docker-compose up --build.
- Access the application at http://localhost.
- Nginx is used as the reverse proxy in front of Flask.

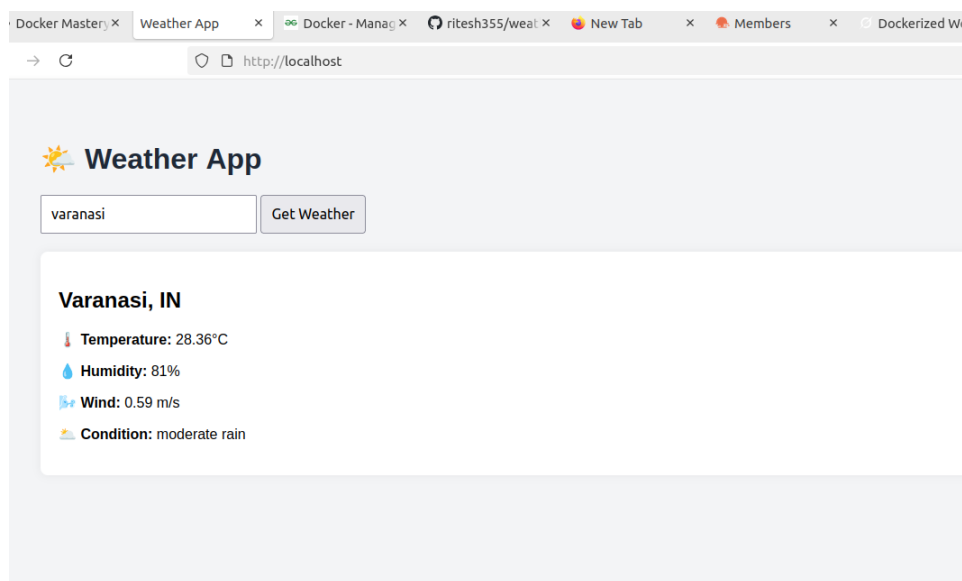


Figure 1.1 — PROJECT-1/assets/result.png

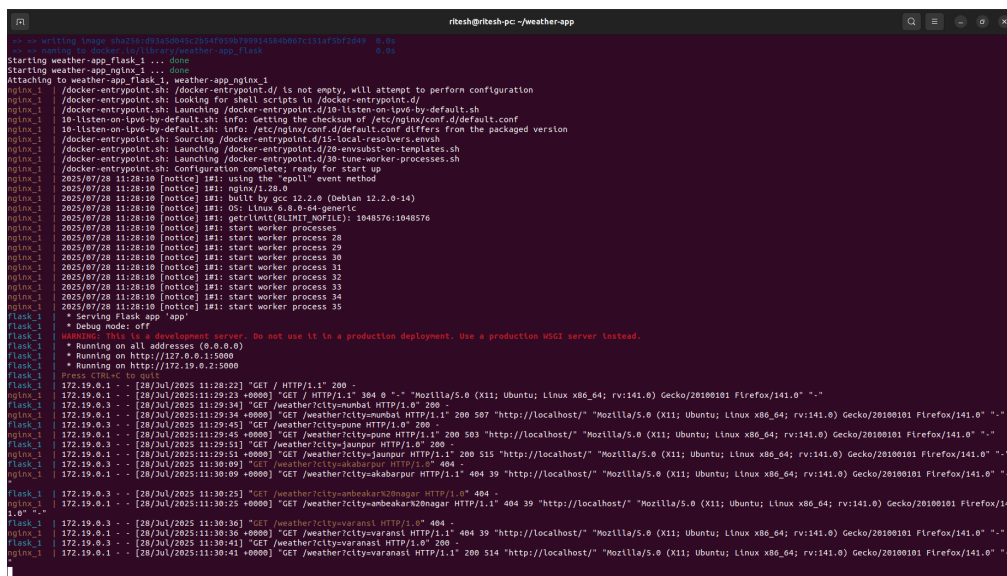


Figure 1.2 — PROJECT-1/assets/live.png

PROJECT-2: Node.js + MongoDB with Docker & Docker Compose

Containerized Node.js/Express application with MongoDB. Docker Compose orchestrates the web and database services and provides a persistent MongoDB volume.

Technology stack

Node.js • Express • MongoDB • Docker • Docker Compose

Documented workflow / setup

- Dockerfile uses node:18-alpine and installs production dependencies.
- docker-compose defines web and mongo services.
- MongoDB uses a persistent volume.
- Application is exposed on port 3000.

No image asset is included in PROJECT-2 in the supplied ZIP; the section is documented from its README.

PROJECT-3: Jenkins CI/CD Pipeline with Docker & Node.js

Jenkins pipeline on AWS EC2 that builds a Node.js application, creates a Docker image, pushes it to Docker Hub and supports GitHub webhook-triggered builds.

Technology stack

Node.js • Jenkins • Docker • AWS EC2 • Docker Hub • GitHub

Documented workflow / setup

- Jenkins is hosted on AWS EC2.
- Pipeline stages include clone, Docker image build, container run and Docker Hub push.
- GitHub webhook integration is documented for automatic builds.
- Docker is used for application packaging and deployment.

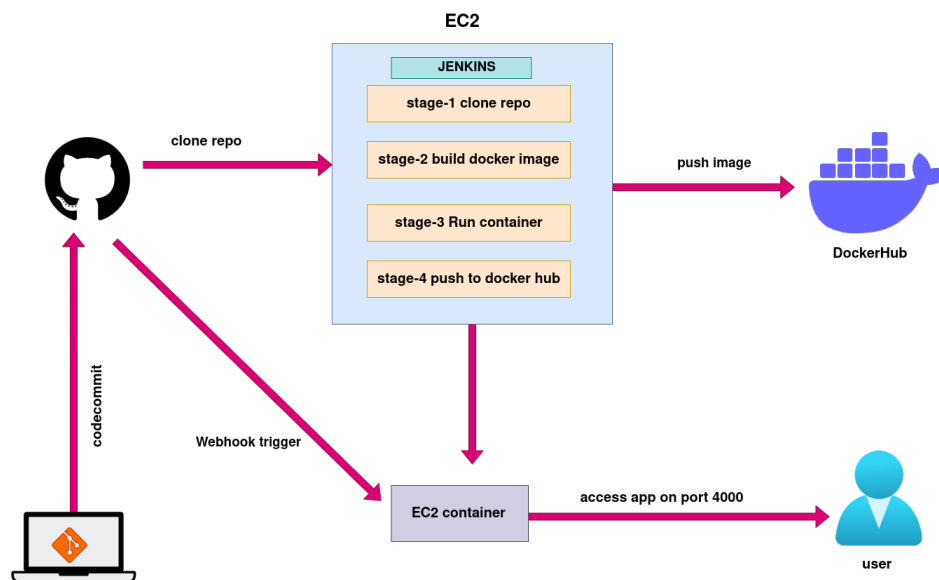


Figure 3.1 — PROJECT-3/images/docker-jenkin.png

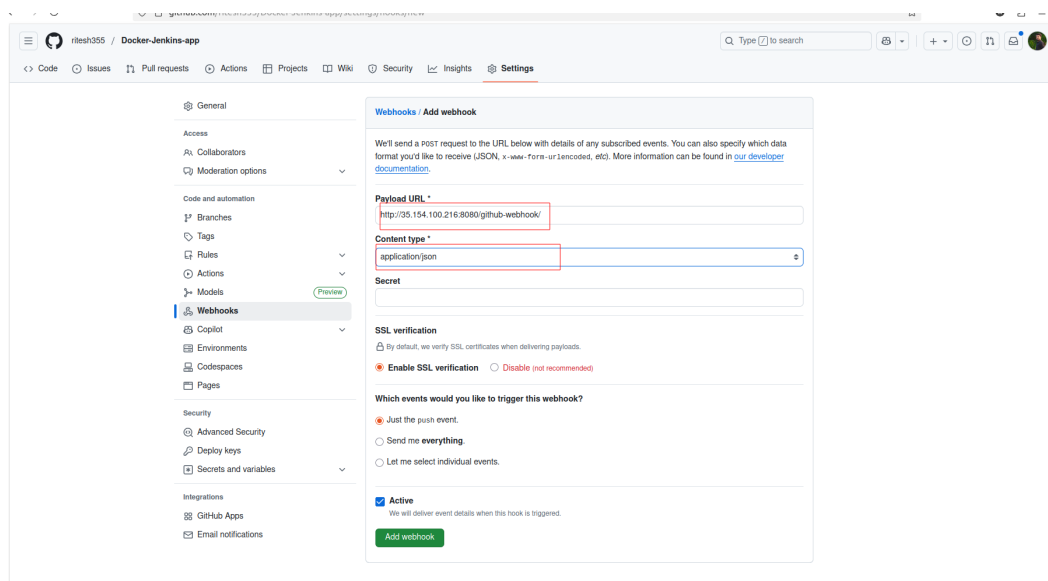


Figure 3.2 — PROJECT-3/images/webhook3.png

PROJECT-4: AWS CI/CD Node.js Application

Node.js application deployed to AWS EC2 using AWS CodePipeline, CodeBuild and CodeDeploy, with GitHub as the source repository.

Technology stack

Node.js • AWS CodePipeline • CodeBuild • CodeDeploy • EC2 • GitHub • S3

Documented workflow / setup

- GitHub is the source stage.
- CodeBuild creates the build artifact.
- CodeDeploy deploys the application to EC2.
- The project documents an S3 artifact bucket and CodeDeploy agent on Ubuntu EC2.

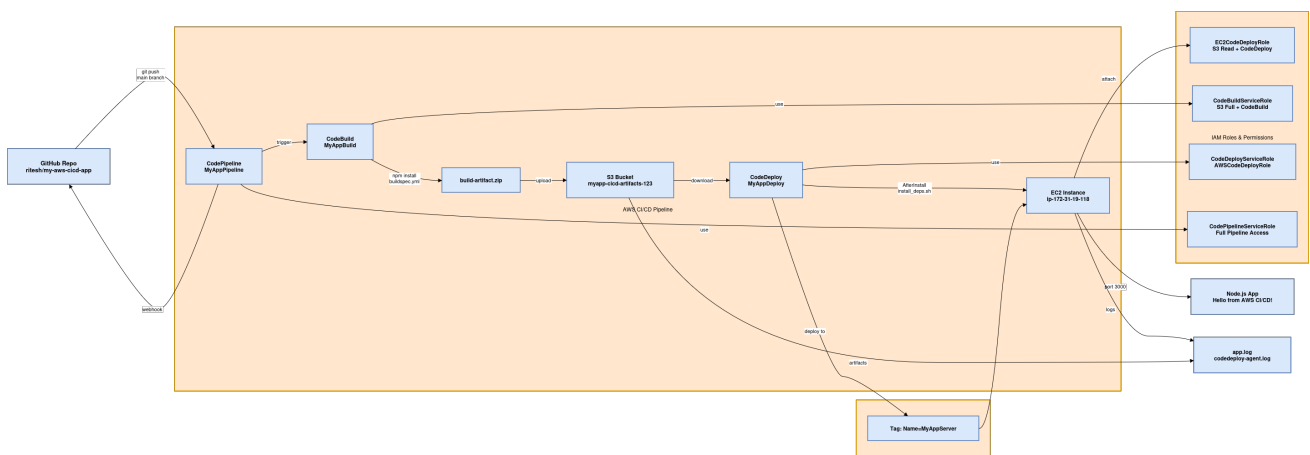


Figure 4.1 — PROJECT-4/assets/flow.png

```
ritesh@ritesh-pc: ~/my-aws-cicd-app
tree
├── appspec.yml
├── buildspec.yml
├── index.js
├── package.json
├── scripts
│   ├── install_deps.sh
│   ├── start_app.sh
│   └── stop_app.sh
└── 2 directories, 7 files
ritesh@ritesh-pc: ~/my-aws-cicd-app$
```

Figure 4.2 — PROJECT-4/assets/repostructure.png

PROJECT-5: Flask + Docker + GitHub Actions CI/CD

A CI/CD project for a Python Flask application, containerized with Docker and automated using GitHub Actions.

Technology stack

Python • Flask • Docker • GitHub Actions • CI/CD

Documented workflow / setup

- GitHub Actions is used as the automation layer.
- The Flask application is containerized with Docker.
- The project includes a workflow diagram in the supplied repository.
- The README presents this as a #100DaysOfDevOps project.

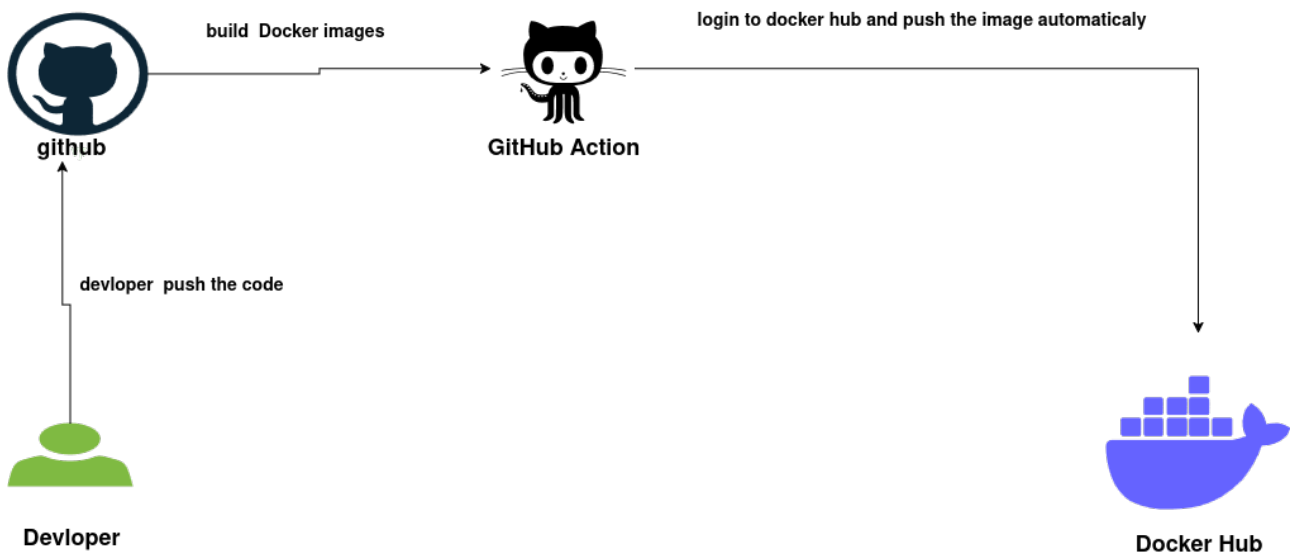


Figure 5.1 — PROJECT-5/ritesh.png

PROJECT-6: Static Website — AWS S3 + CloudFront + CI/CD

Static website hosted on Amazon S3, delivered globally through CloudFront and deployed automatically with GitHub Actions. The README describes Origin Access Control and cache invalidation.

Technology stack

AWS S3 • CloudFront • GitHub Actions • OAC • CI/CD

Documented workflow / setup

- CloudFront is the delivery layer in front of S3.
- Origin Access Control keeps the S3 bucket private to CloudFront.
- GitHub Actions uploads changes and performs cache invalidation.
- The documented request flow is User → CloudFront → S3 on cache miss.

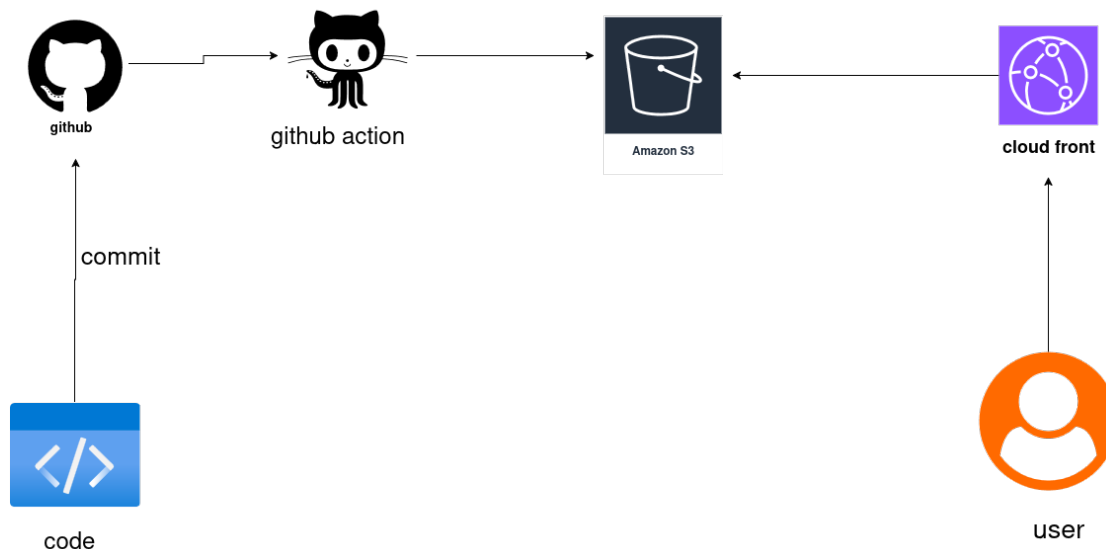


Figure 6.1 — PROJECT-6/screenshots/cloudDiagram.drawio.png



Figure 6.2 — PROJECT-6/screenshots/gallery-preview.png

PROJECT-7: AWS DynamoDB Flask Task Manager

Flask CRUD application for tasks stored in AWS DynamoDB, with a Bootstrap UI for adding, editing and deleting tasks.

Technology stack

Flask • AWS DynamoDB • Bootstrap • Python • CRUD

Documented workflow / setup

- DynamoDB table name: Users.
- Primary key: TaskID (String).
- The UI supports Add, Edit and Delete task operations.
- Local setup uses a Python virtual environment and Flask on port 5000.

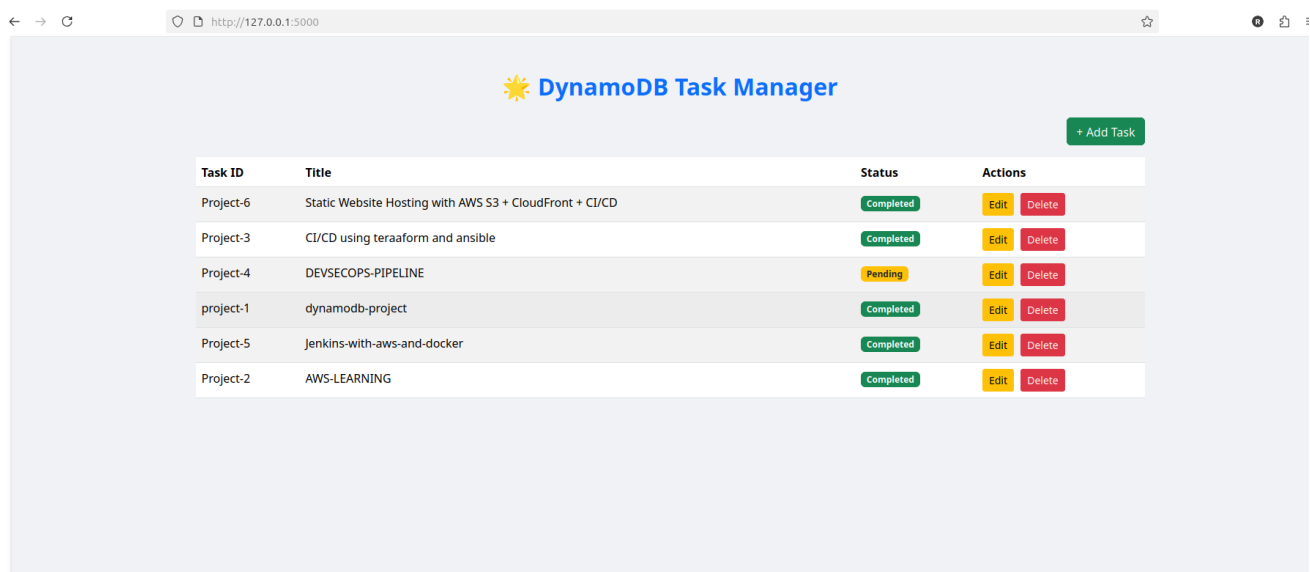


Figure 7.1 — PROJECT-7/images/project.png

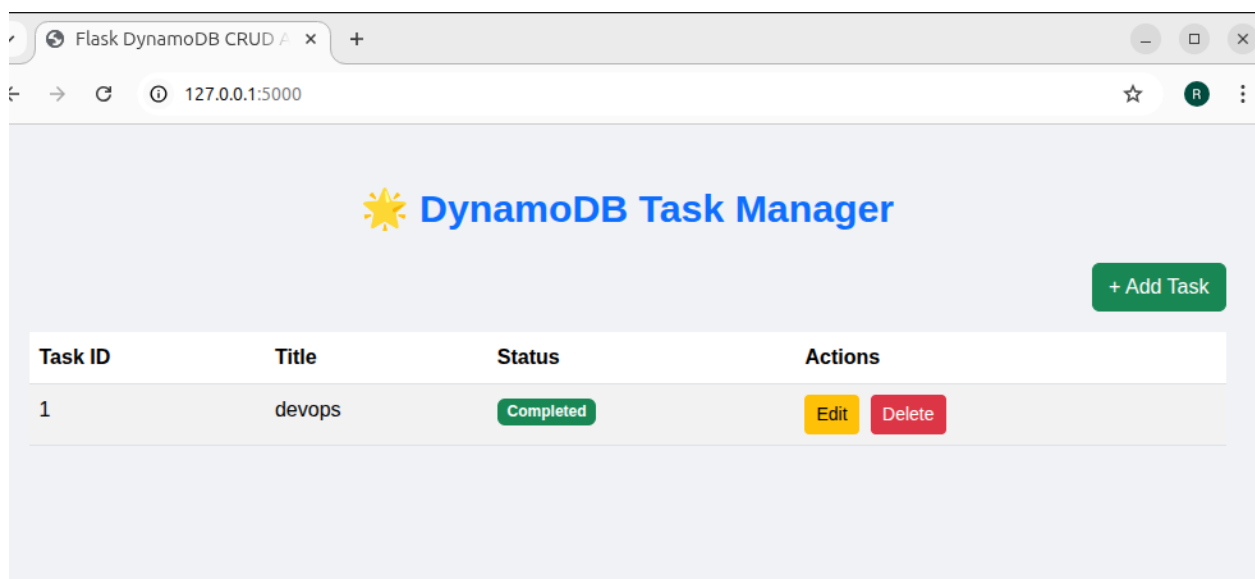


Figure 7.2 — PROJECT-7/images/crud.png

PROJECT-8: Serverless CI/CD Pipeline using AWS Lambda

Event-driven, serverless CI/CD pipeline where a GitHub webhook reaches API Gateway, Lambda starts CodePipeline, CodeBuild builds/tests, S3 stores artifacts and CloudFront serves the site.

Technology stack

GitHub • API Gateway • Lambda • CodePipeline • CodeBuild • S3 • CloudFront • CloudWatch • IAM

Documented workflow / setup

- GitHub push triggers a webhook to API Gateway.
- Lambda starts the CodePipeline execution.
- CodeBuild builds/tests and S3 stores the deployment artifacts.
- CloudFront delivers the deployed static website; IAM controls permissions.

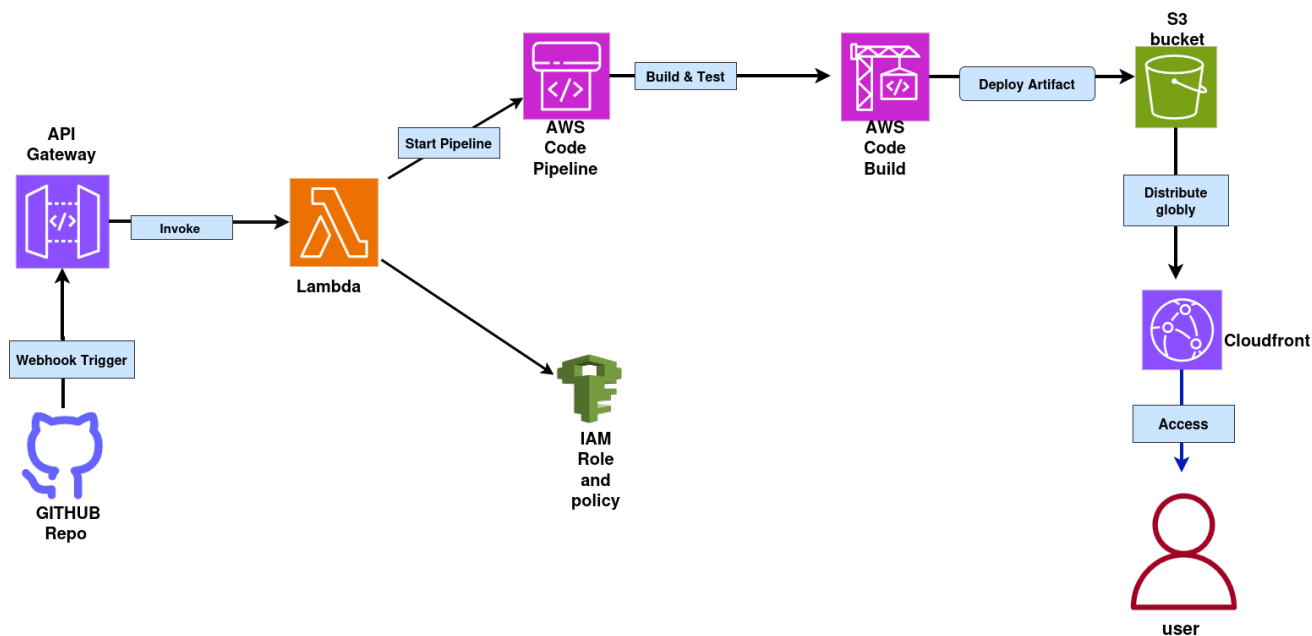


Figure 8.1 — PROJECT-8/assets/very.png

PROJECT-9: Next.js Portfolio — DevOps Ready

Next.js portfolio containerized with Docker, deployed on AWS EC2 and distributed through CloudFront. GitHub Actions builds, pushes to Docker Hub and deploys to EC2. The README also describes Prometheus, Grafana and cAdvisor monitoring.

Technology stack

Next.js • React • Docker • Docker Hub • AWS EC2 • CloudFront • GitHub Actions • Prometheus • Grafana • cAdvisor

Documented workflow / setup

- Next.js application is containerized using Docker.
- GitHub Actions builds the app, pushes an image to Docker Hub and deploys to EC2 via SSH.
- CloudFront is used for global delivery.
- The README documents Prometheus, Grafana and cAdvisor as the monitoring stack.

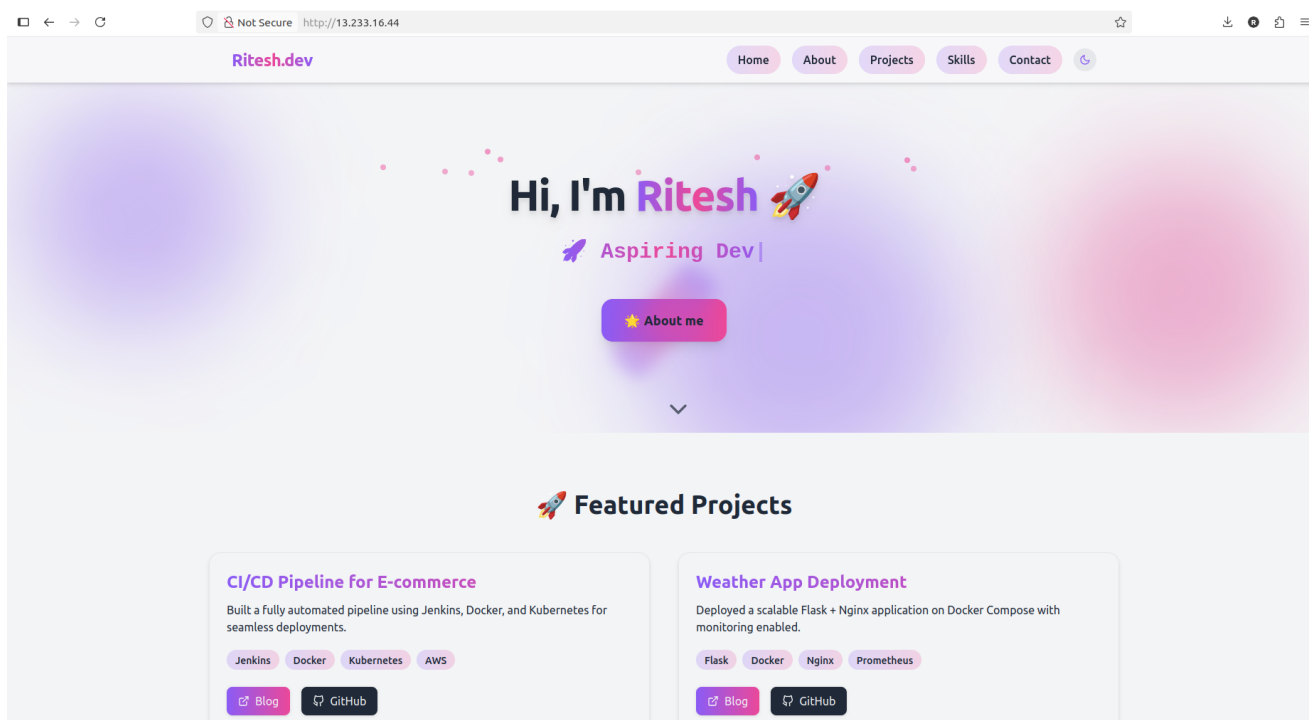


Figure 9.1 — PROJECT-9/assets/home.png

PROJECT-10: DevSecOps CI/CD with Terraform, Ansible & Jenkins

Full DevSecOps pipeline for a Flask app: Docker build, Trivy scanning, AWS ECR image storage, Terraform infrastructure, Ansible configuration/deployment, Jenkins orchestration, Ansible Vault and Prometheus/Grafana monitoring.

Technology stack

Jenkins • Docker • AWS ECR • Terraform • Ansible • Trivy • Ansible Vault • Prometheus • Grafana

Documented workflow / setup

- Terraform provisions EC2, security groups and IAM resources.
- Ansible configures servers and deploys the application.
- Jenkins orchestrates build, scan, push and deployment.
- Trivy scans container images; ECR stores images; Prometheus/Grafana provide monitoring.

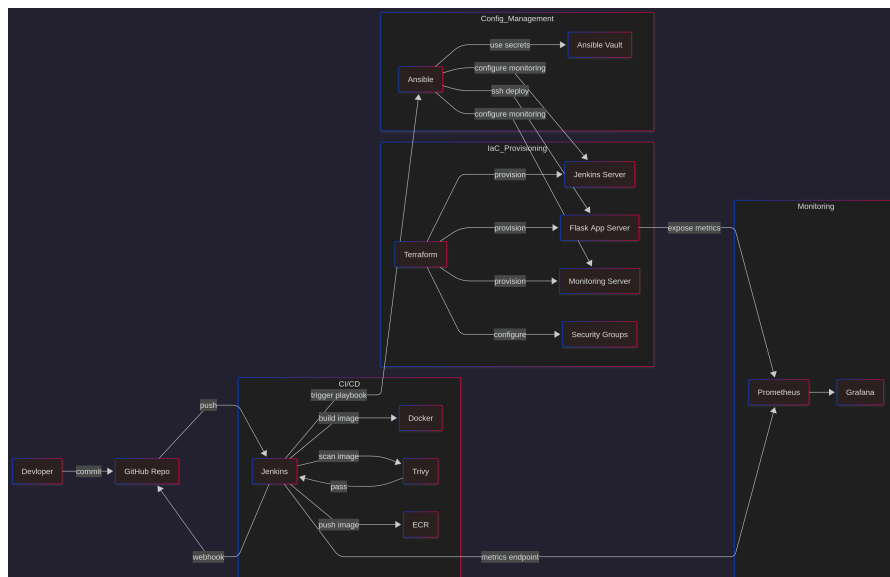


Figure 10.1 — PROJECT-10/images/work.png

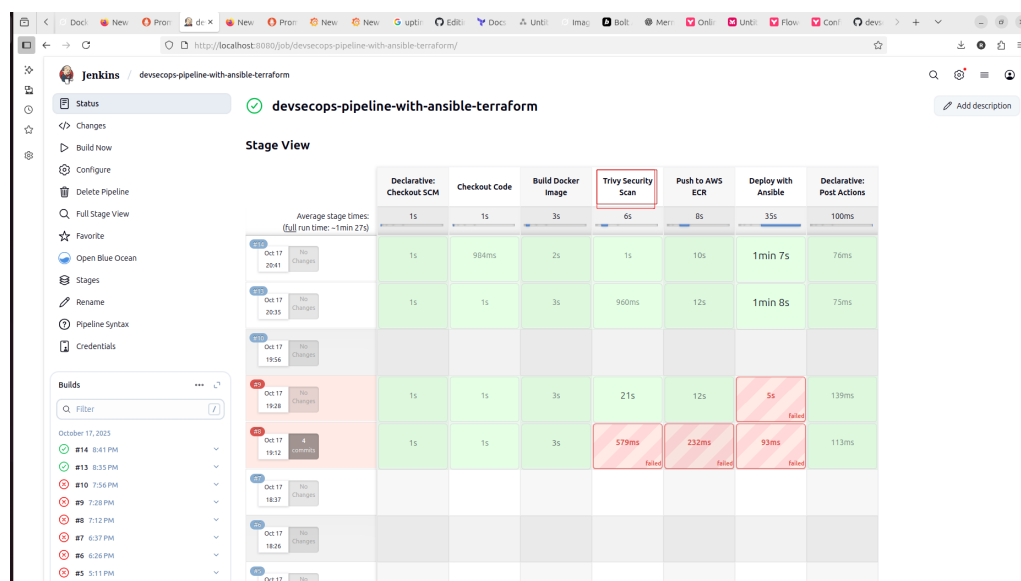


Figure 10.2 — PROJECT-10/images/trivy.png

11. Cross-Project DevOps Concepts

Across the supplied projects, the documentation demonstrates recurring DevOps practices:

- Containerization with Docker and multi-container orchestration with Docker Compose.
- CI/CD automation using Jenkins, GitHub Actions and AWS CodePipeline/CodeBuild/CodeDeploy.
- AWS cloud deployment using S3, CloudFront, EC2, DynamoDB, Lambda, API Gateway and ECR.
- Infrastructure and configuration automation using Terraform and Ansible.
- Security scanning and secret-management concepts using Trivy, IAM and Ansible Vault.
- Observability with Prometheus, Grafana, CloudWatch and cAdvisor in the projects that document them.

12. Repository Submission Notes

The supplied ZIP contains PROJECT-1 through PROJECT-10 plus a root README.md. The screenshots in this document are taken from the image assets included in those project folders. Where a project does not include a screenshot asset, the documentation explicitly notes that and relies on the supplied README.

13. Conclusion

This portfolio covers a progression from basic containerized applications to AWS-based CI/CD, serverless automation, cloud deployment, monitoring and a full DevSecOps pipeline. The documentation preserves the project structure and terminology present in the supplied source ZIP.